Ders 11 Genişletilmiş Notu: Seminer Sunumları: Bayesçi Model Sıkıştırma, Transformer, FastText ile Metin Kodlama, Otokodlayıcı, CodeComp, Shannon-Fano/Huffman ve DWT-CNN

Ders 11 Genişletilmiş Notu: Seminer Sunumları: Bayesçi Model Sıkıştırma, Transformer, FastText ile Metin Kodlama, Otokodlayıcı, CodeComp, Shannon-Fano/Huffman ve DWT-CNN

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Yapısı
- 2. Sunum 1: Doğal Dil İşleme için Bayesçi Sıkıştırma
- 3. Sunum 2: Transformer ile Kayıpsız Veri Sıkıştırma
- 4. Sunum 3: Derin Sinir Ağını Kayıplı Sıkıştırma Aşaması Olarak Kullanan Metin Kodlama
- 5. Sunum 4: Otokodlayıcı ile Kayıplı Görüntü Sıkıştırma
- 6. Sunum 5: FOTA için Program Kodu Sıkıştırma (CodeComp)
- 7. Sunum 6: Küçük Mobil Cihazlarda Shannon-Fano ve Huffman
- 8. Sunum 7: Sualtı Nesnelerin İnterneti için DWT-CNN ile Görüntü Sıkıştırma
- Örnek Sorular

1. Dersin Yapısı

Bu ders, bir önceki dersteki gibi öğrenci seminer sunumlarına ayrıldı. Her sunumda bir öğrenci veri sıkıştırma ile ilgili yayımlanmış bir makaleyi özetledi. Hoca her sunumdan önce makalenin konusunu ve yılını söyledi, sunumdan sonra sorular sordu ve yorum yaptı. Toplam yedi sunum yapıldı. Konular sırasıyla şunlardır: tekrarlayan sinir ağlarının Bayesçi yöntemle küçültülmesi, Transformer ile kayıpsız metin sıkıştırma, kelime gömmeleri ve Huffman kodlamasıyla iki aşamalı metin kodlama, otokodlayıcı ile kayıplı görüntü sıkıştırma, cep telefonu yazılım güncellemesi için program kodu sıkıştırma, akıllı kartlarda Shannon-Fano ve Huffman karşılaştırması, sualtı görüntüleri için dalgacık dönüşümü ve CNN birleşimi.

Sunumların bir kısmı İngilizce yapıldı ve altyazı bu bölümlerde (özellikle 00:20:00-00:38:00 ve 01:45:00-01:57:00) çoğu yerde anlamsız tekrara düştü. Bu sunumların anlatımı büyük ölçüde ekrandaki slaytlardan kuruldu.

2. Sunum 1: Doğal Dil İşleme için Bayesçi Sıkıştırma

İlk sunumda bir öğrenci, hocanın belirttiğine göre 2018 yılına ait “Bayesian Compression for Natural Language Processing” çalışmasını anlattı.

2.1 Motivasyon

Doğal dil işleme (NLP), doğal dillerin kurallı yapısını çözümleyerek anlamayı ve yeniden üretmeyi amaçlar. Sunan öğrenci, bankalardaki sohbet botları ve telefondaki sesli asistanlar gibi örneklerle bu alanın önemini anlattı. Konuşma tanıma ve soru cevaplama gibi görevlerde en güçlü modellerden biri tekrarlayan sinir ağlarıdır (RNN). RNN'ler çok sayıda parametre içerir; sınırlı belleği olan bir cihazda kullanılabilmeleri için sıkıştırılmaları gerekir.

Çalışmanın iki ana fikri vardır:

- **Sparse VD'nin RNN'lere uygulanması:** Budama gibi yöntemlerde bazı parametrelerin elle ayarlanması gerekir. Sparse VD (seyrek varyasyonel dropout) bu ayarı eğitim sırasında kendiliğinden yapar.
- **Kelime dağarcığının seyreltilmesi:** NLP görevlerinde doğru tahmin için gereksiz olan ağırlıkların çoğu gömme (embedding) katmanında yoğunlaşır. Bu katmanı seyreltmek için çarpımsal ağırlıklar eklenir. Eğitim sırasında sıfırlanan ağırlıklar, ihtiyaç olmayan kelimelerin atılması anlamına gelir; böylece RNN'in seyreklik düzeyi daha da artar.

2.2 İlgili çalışmalar

Slayta göre RNN boyutunu azaltmanın üç yönelimi vardır (ekrandan, 00:10:00):

- Ağırlık matrislerinin tahmini
- Ağırlık matrislerinin hassasiyetinin azaltılması
- Ağırlık matrislerinin seyreltilmesi

Bu çalışma üçüncüsüne odaklanır. Bu yönelimdeki en yaygın yaklaşım budamadır (pruning): ağırlıklar belirli bir eşikte kesilir. Slaytta örnek olarak ICLR 2017'de yayımlanan "Exploring Sparsity in Recurrent Neural Networks" çalışması gösterildi; orada eşik, frekansı ve hızı kontrol eden birkaç parametreyle seçilir. Seyreltmenin bir başka yolu Bayesçi sinir ağlarıdır: ağırlıklar rastgele değişken olarak ele alınır, eğitimde önsel (prior) dağılım sonsal (posterior) dağılıma dönüştürülür. Sunan öğrenciye göre Bayesçi seyreltme hem daha az hiperparametre ister hem de daha yüksek sıkıştırma oranı verir.

2.3 Yöntem: Sparse VD

Gösterimde x girdi dizisi, y gerçek çıktı, ω bütün RNN ağırlıklarıdır. Sparse VD'de ağırlıkların önsel dağılımı log-uniform, sonsal dağılımı tam ayrışık (fully factorized) normal dağılım olarak alınır. Sonsalın bulunması, varyasyonel alt sınırın en iyilenmesine denktir. Bu ifadenin iki terimi vardır:

- Birinci terim, sonsaldan alınan bir örnekle tahmin edilen veri uyumu terimidir.
- İkinci terim, sonsalı önsele yaklaştıran ve seyrekliği tetikleyen bir düzenleyicidir (regularizer).

RNN'in ileri beslemeli ağlardan farkı, her zaman adımında aynı ağırlıkları kullanmasıdır. Çalışma, ağırlıkları tek tek örneklemek yerine yerel yeniden parametreleme hilesini (Local Reparameterization Trick, LRT) kullanır; bu, eğitimi hızlandırır. Özetle: ağırlıklar örneklenir, RNN uygulanır, gradyanlar hesaplanır; test aşamasında ortalama ağırlıklar kullanılır. Bileşenlerin çoğu sıfıra yaklaşır; sinyal-gürültü oranı düşük olan ağırlıklar tamamen atılır.

> **Ek bilgi (derste anlatılmadı):** Altyazıda "King Mazda" diye geçen ad büyük olasılıkla Kingma'dır; varyasyonel dropout ve yerel yeniden parametreleme hilesi Kingma ve arkadaşlarının 2015 tarihli çalışmasından gelir. Sparse VD ise büyük olasılıkla Molchanov ve arkadaşlarının 2017 tarihli çalışmasıdır.

2.4 Deneyler ve sonuçlar

Yöntem iki görevde denendi: metin sınıflandırma ve dil modelleme. Her görevde üç model karşılaştırıldı: düzenleme içermeyen temel model (Original), Sparse VD ve çarpımsal ağırlıklı Sparse VD-Voc. Veri setleri slaytta gösterildi (ekrandan, 00:12:40): ikili sınıflandırma için IMDb (Large Movie Review Dataset), dört sınıflı sınıflandırma için AG's corpus of news articles, dil modelleme için Penn Treebank. Sunan öğrenciye göre sınıflandırmada 300 birimlik gömme katmanı ve LSTM, dil modellemede 256 gizli birimli LSTM kullanıldı.

Metin sınıflandırma sonuçları (ekrandan, 00:13:20):

Görev	Yöntem	Doğruluk %	Sıkıştırma	Dağarcık	Nöron $\tilde{x}$ - h
IMDb	Original	84,1	1x	20000	300 - 128
IMDb	SparseVD	85,1	1135x	4611	16 - 17
IMDb	SparseVD-Voc	83,6	12985x	292	1 - 8
AGNews	Original	90,6	1x	20000	300 - 512
AGNews	SparseVD	88,8	322x	5727	179 - 56
AGNews	SparseVD-Voc	89,2	469x	2444	127 - 32

Dil modelleme sonuçları (ekrandan, 00:13:20):

Görev	Yöntem	Geçerleme	Test	Sıkıştırma	Dağarcık	Nöron h
Char PTB (bit/karakter)	Original	1,498	1,454	1x	50	1000
Char PTB	SparseVD	1,472	1,429	4,2x	50	431
Char PTB	SparseVD-Voc	1,4584	1,4165	3,53x	48	510
Word PTB (perplexity)	Original	135,6	129,5	1x	10000	256
Word PTB	SparseVD	115,0	109,0	14,0x	9985	153
Word PTB	SparseVD-Voc	126,3	120,6	11,1x	4353	207

Metin sınıflandırmada önemli bir doğruluk kaybı olmadan çok yüksek sıkıştırma oranları elde edildi; Sparse VD-Voc doğruluğu korurken oranı daha da artırdı. Sunan öğrenciye göre bu kadar yüksek oranların nedeni kelime dağarcığının seyrekliğidir. Dil modellemede 50 karakterlik ve 10.000 kelimelik dağarcıklar kullanıldı. Burada sınıflandırmadaki kadar yüksek oranlar alınamadı: karakter düzeyinde her karakter önemli olduğu için dağarcık seyreltilemedi (50'den 48'e), kelime düzeyinde ise dağarcığın yarısından fazlası elendi (10000'den 4353'e). Dil modellemede bütün kelimeler önemli olduğu için seyreltme görevi zorlaştı ve kalite düştü.

2.5 Hocanın soruları ve yorumu

Hoca sunumu net bulduğunu söyledi, ama çalışmanın amacını sorguladı.

Soru: Seyreltme sırasında bazı kelimeler atılıyor. Burada amaç sıkıştırma mı, sınıflandırma mı? Sıkıştırma yapılıyorsa geri dönüş olmalı, ama atılan kelimeler kaybolmuş oluyor. **Cevap:** Sunan öğrenciye göre yöntem, sınıflandırmaya yol göstermeyen nötr kelimeleri gereksiz sayıp atar ve sınıflandırma kalitesini düşürmez; amaç sınıflandırırken modeli sıkıştırmaktır. Hoca, çalışmanın tamamen bir sınıflandırma problemiyle ilgili olduğunu, sıkıştırma kısmının da metni değil modeli küçülttüğünü söyledi.

Soru: Dil modelinde eksik kalan kelimeler sonradan dil modeli yardımıyla bulunuyor mu? **Cevap:** Sunan öğrenciye göre model, önceki ve sonraki kelimelerden yararlanarak sonraki karakteri ya da kelimeyi tahmin eder; karakter düzeyinde ve kelime düzeyinde iki veri setiyle çalışılmıştır.

Soru: Modelde kaç gizli katman ve kaç nöron var? **Cevap:** Sınıflandırmada 300 birimlik gömme katmanı ve ardından LSTM katmanı, dil modellemede 256 gizli birimli LSTM.

3. Sunum 2: Transformer ile Kayıpsız Veri Sıkıştırma

İkinci sunumda bir öğrenci, hocanın belirttiğine göre 2020 yılına ait "Lossless Data Compression with Transformer" çalışmasını İngilizce anlattı. Hoca, Transformer'ların güncel ve popüler bir konu olduğunu söyledi.

3.1 Kayıpsız sıkıştırma ve istatistiksel yöntemler

Slayttaki tanıma göre kayıpsız sıkıştırma, orijinal verinin tam olarak geri kurulmasına izin veren sıkıştırma algoritmaları sınıfıdır (ekrandan, 00:20:00). İstatistiksel kayıpsız sıkıştırmada yaygın yaklaşım PAQ ailesidir; bit düzeyinde çalışan PAQ8 her tür veriye uygulanabilir. Bu yaklaşımın iki parçası vardır: geçmişe bakarak bir sonraki sembolün olasılığını kestiren bir tahminci ve kestirilen olasılıkla sembolü kodlayan entropi kodlayıcı (aritmetik kodlamaya benzer). Sıkıştırmadaki iyileşme tahmincinin iyileşmesinden gelir.

PAQ8'in tahmincisi elle tasarlanmış (handcrafted) özelliklere dayanır. Çalışmanın fikri, dizi modelleme ile makine öğrenmesi arasındaki yakın ilişkiyi kullanıp tahminciyi tamamen sinir ağı tabanlı bir modelle, Transformer ile değiştirmektir. Sunan öğrenciye göre bu problem, bir dil modelinin çevrimiçi (online) öğrenilmesi problemine çok benzer. Daha önce LSTM kullanan sinir ağı yaklaşımları (ve CMIX) vardır; bütün bu yaklaşımlar entropi kodlaması olarak aritmetik kodlamayı kullanır.

3.2 Uygulama

Veri seti, metin sıkıştırma için standart olan enwik8'dir: İngilizce Vikipedi'nin 100 MB'lık bir kısmı. Ön işlem olarak CMIX ön işlemcisi kullanılır: bütün büyük harfler, önüne bir kaçış (escape) kodu konarak küçük harfe çevrilir.

3.3 Parametrelerin sıkıştırma oranına etkisi

Model boyutu (ekrandan, 00:27:20): Grafikte bit/karakter değeri, parametre sayısı, katman sayısı ve verinin yüzdesine göre üç boyut (dim = 256, 512, 1024) için çizilmiştir.

- Sol: enwik8'in tamamında büyük modeller daha iyi sıkıştırma oranı verir.
- Orta: 12 katmandan derin modeller oranı iyileştirmez.
- Sağ: 12 katmanlı modellerde anlık sıkıştırma oranı (son 1 milyon karakter üzerinden ortalama), büyük modellerin sıkıştırmanın başında bile daha iyi olduğunu gösterir.

Bağlam büyüklüğü (ekrandan, 00:30:00): Transformer'da bağlam büyüklüğü, modelin sonraki sembolü tahmin ederken kullanabileceği önceki gizli durumların en büyük sayısıdır. Gözlemler:

- Daha büyük bağlam genelde daha iyi performans verir.
- Sıkıştırmanın başında kısa bağlamlı modeller daha iyidir.
- Sonraki karakteri tahmin etmek için en yararlı bilgi genelde birkaç önceki karakterdedir.
- Daha büyük bellek ağa daha çok bilgi sağlar.

Öğrenme hızı: Öğrenme hızı, sinir ağının yakınsama hızını önemli ölçüde etkileyen bir parametredir. Yüksek sabit öğrenme hızı sıkıştırmanın başında daha iyi oran verir, düşük öğrenme hızı ise ancak sonda iyi sonuç verir. Doğrusal azalan (linear decay) çizelge, sıkıştırmanın her aşamasında en iyi sabit hızdan daha iyidir.

En iyileyici parametreleri: Transformer'ı eğiten en iyileyicinin (optimizer) parametreleri sıkıştırma oranını hassas biçimde etkiler; sunan öğrenciye göre küçük değerler büyük değerlerden daha iyi sıkıştırma verir.

Ön işlem ve n-gram bilgisi: Girdi belirteçlerine n-gram bilgisi eklenince sıkıştırmanın bütün aşamalarında oran iyileşir; bu, n-gram'ı temsil eden vektör sayısına (K_{max}) bağlıdır.

Veriyi yeniden ziyaret: Modelin birden çok güncellemesini hesaplayabilmek için sıkıştırılan verinin tamamı saklanır ve yeniden ziyaret edilir; bu, olasılık modelinin kalitesini artırır.

3.4 Karşılaştırma ve sonuç

Başka yaklaşımlar kod çözücüye bir sözlük göndermek zorunda olduğu için karşılaştırma zordur. Bu yüzden farklı yöntemlerde kullanılan sözlüğün gzip -9 ile sıkıştırıldıktan sonraki boyutu hesaba katılır. Ağları eğitmek için kullanılan kodun da kod çözücüye gönderilmesi gerekir, ama boyutu çok küçüktür (ekrandan, 00:36:30). Sonuç olarak Transformer tabanlı yaklaşım, elle tasarlanmış özellikler kullanmadan daha iyi sonuç verir.

3.5 Hocanın yorumu

Hoca, çalışmayı büyük bir Transformer'ın hiperparametreleri değiştirildiğinde başarının nasıl değiştiğini inceleyen bir çalışma olarak özetledi.

4. Sunum 3: Derin Sinir Ağını Kayıplı Sıkıştırma Aşaması Olarak Kullanan Metin Kodlama

Üçüncü sunumda bir öğrenci, hocanın belirttiğine göre 2020 yılına ait "Source Coding for Text Transmission using a Deep Neural Network as a Lossy Compression Stage" çalışmasını anlattı (ekrandan, 00:40:00).

4.1 Amaç

Telekomünikasyon ağlarında artan trafik, iletilecek bilgiyi yüksek oranda sıkıştıran kodların aranmasını gerektirir. Geleneksel algoritmaların sıkıştırma oranları sınırlı olduğu için yeni teknolojilerle bu oranları artırma çalışmaları başlamıştır. Bu çalışma derin öğrenmeyi iki aşamalı bir metin sıkıştırma çözümünde kullanır: ilk aşamada FastText mimarisiyle kayıplı bir metin sıkıştırma yapılır, ikinci aşamada geleneksel kanonik Huffman koduyla veri kodlanır.

Kelime gömmeleri (embedding), anlamca benzer kelimelerin birbirine yakın, ilişkisiz kelimelerin birbirinden uzak olduğu bir kelime vektör uzayı sağlar. Sıkıştırma burada bilginin bir iletişim kanalından iletilmesi için gereken bit miktarını azaltmak demektir.

4.2 Önerilen yöntem

Slayttaki akış (ekrandan, 00:42:30) şöyledir:

```
Kaynak kodlama:
  Metin -> Kelime çıkarma -> FastText ağı -> (Yeni kelime, Ağdaki uzaklık)
        -> Sıkıştırılmış metin -> Huffman kodu -> İkili dizi -> KANAL
Kaynak çözme:
  İkili dizi -> Başlığı çözümle -> Huffman tablosu
  İkili dizi -> Sıkıştırılmış ikili veri -> Sıkıştırılmış metni geri kur
        -> (Kelimeler, Ağdaki uzaklık) -> FastText ağı -> Orijinal kelime -> Metin
```

Girdi metni kelimelere ayrılır, her kelime FastText ağından geçirilerek vektöre çevrilir. Her kelime için kosinüs benzerliğine göre en yakın K komşu kelime bulunur. Bunlar arasından en az karakterli olan seçilir ve yanına kaçıncı komşu olduğu yazılır. Örneğin "automobile" kelimesinin en yakın komşuları arasında "car" vardır; "car" ve komşu sırası gönderilir. Bu sıkıştırılmış metin daha sonra Huffman ile ikili diziye çevrilir.

Kod çözmede önce Huffman ile sıkıştırılan ikili veri, kodlama sırasında oluşan kod sözcüğü tablosuyla çözülür. Ardından her kelime ve komşu sırası FastText ağına verilir; ağ, verilen kelimenin o sıradaki komşusu olan orijinal kelimeyi bulur. Başlık (header) bilgisinde sembollerin olasılıkları azalan sırada tutulur; bu, Huffman tablosunun çözümlenmesi için gereklidir.

4.3 Hocanın soruları

Soru: FastText kelimelere vektör atıyor. "car" 3 karakterden oluşuyor, ama onun vektörü belki 100 boyutlu. 3 byte yerine 100 boyutlu bir vektör göndermek veriyi büyütmez mi? **Cevap:** Vektör gönderilmez. Vektörler yalnız benzerliği bulmak için kullanılır. "automobile" kelimesinin vektörüne kosinüs benzerliğiyle en çok benzeyen K kelime getirilir (örneğin $K = 5$ ise 5 kelime), bunlardan en az karakterli olan seçilir ve yanına kaçıncı komşu olduğu yazılır. Hoca bu fikri beğendiğini söyledi.

Soru: Geri dönüş nasıl yapılıyor? **Cevap:** Huffman ile çözülen metinde kelime ve sıra numarası ayrı ele alınır; komşuluk bilgisiyle orijinal kelime bulunur.

Soru: Eklenen konum bilgileri de veriyi biraz artırmıyor mu? **Cevap:** Hoca kendi sorusunu şöyle yanıtladı: konum bilgisi yer kaplar, ama "automobile" 10 karakterdir ve onun yerine yaklaşık 4 byte gider. Sunan öğrenci, başlıklı ve başlıksız deneylerde bu etkinin görüldüğünü ekledi.

4.4 Deneysel sonuçlar

Çalışma üç açıdan değerlendirilmiştir: sıkıştırma oranı, K parametresinin hata oranına etkisi ve kayıplı sıkıştırmanın iletilen bilginin içeriğini ne kadar bozduğu. Sunan öğrenciye göre sonuçlar 5-10 kelime uzunluğundaki 50 farklı cümle üzerinden alınmıştır.

Sıkıştırma oranı (ekrandan, 00:47:30): Dört deney yapılmıştır. Grafikte dikey eksen cümlenin normalize boyutu, yatay eksen kelime sayısıdır.

- Yalnız FastText: yaklaşık 0,67-0,77.
- FastText + Huffman (başlıklı): 5 kelimede yaklaşık 1,04 (yani sıkıştırma yok), 15 kelimede yaklaşık 0,70, 100 kelimede yaklaşık 0,45.
- FastText + Huffman (başlıksız): yaklaşık 0,39-0,42, en iyi sonuç.
- Yalnız Huffman (başlıksız): yaklaşık 0,56-0,57.

Başlık bilgisi kullanıldığında kısa cümlelerde başlığın yükü yüzünden sıkıştırma çok düşüktür; kelime sayısı arttıkça sıkıştırma iyileşir. Yalnız FastText ya da yalnız Huffman tek başına yeterli değildir; ikisi birlikte daha iyidir.

K **parametresi** (ekrandan, 00:48:30): Grafikte kelime hata oranı cümle uzunluğuna göre çizilmiştir. Lejant: $K = 5$ - Compress 82, $K = 10$ - Compress 77, $K = 20$ - Compress 74, $K = 30$ - Compress 73, $K = 50$ - Compress 70. $K = 5$ için hata bütün uzunluklarda sıfırdır. K ve cümle uzunluğu arttıkça hata hızla artar; 40 kelimede $K = 50$ için hata 3'ün üzerindedir.

Kayıplı kodlama örneği (ekrandan, 00:50:00):

K	Durum	Cümle
-	Orijinal	what is your phone number
5	Sıkıştırılmış	how3is1my2phone1number1
5	Açılmış	what is your phone number
30	Sıkıştırılmış	it40is1my2fax12
30	Açılmış	what is your phone
50	Sıkıştırılmış	it40is1my2fax12
50	Açılmış	what is your phone
-	Orijinal	paper money has been used for over a thousand years
5	Sıkıştırılmış	paper1cash2is5has2used1of4over1a1dozen8days4
5	Açılmış	paper money has been used for over a thousand years
30	Sıkıştırılmış	ink7cash2is5be9used1a33in26a1ten92yrs3
30	Açılmış	paper money has been used for over a thousand years
50	Sıkıştırılmış	ink7cash2is5be9use8a33a90a1ten92yrs3
50	Açılmış	paper money has been used for over a thousand years

Sıkıştırılmış biçimde her kelimenin yerine daha kısa bir komşu ve sıra numarası yazılır: örneğin "what" yerine "how3", "your" yerine "my2". İlk cümlede K büyüyünce bilgi kaybı olmuş, "number" kaybolmuştur. İkinci cümlede K'nin artması kayba yol açmamış, sıkıştırmayı artırmıştır. Sunan öğrenciye göre kayıplı modda K büyüdükçe anlamsal bozulma ortaya çıkabilir.

4.5 Sonuç ve gelecek çalışmalar

Huffman kodlamasından önce NLP çözümü kullanılarak mesaj boyutu yaklaşık %30 azaltılmıştır. Başlıklı FastText + Huffman çözümü kısa cümlelerde düşük sıkıştırma verir, kelime sayısı arttıkça oran iyileşir. K parametresi, kayıp oranının ayarlanmasını sağlar. Gelecek çalışma olarak FastText'ten sonraki ikinci aşamada başka kaynak kodlama yöntemlerinin denenmesi ve askeri iletişim sistemleri için kriptolojik işlemlerin eklenmesi planlanmıştır.

Hoca çalışmayı değişik ve ilginç bulduğunu, böyle bir fikri ilk kez duyduğunu söyledi.

5. Sunum 4: Otokodlayıcı ile Kayıplı Görüntü Sıkıştırma

Dördüncü sunumda bir öğrenci, hocanın belirttiğine göre 2018 yılına ait, derin ağlarla (evrişimli otokodlayıcı) kayıplı görüntü sıkıştırma çalışmasını anlattı.

5.1 Motivasyon

Görüntüler yer kazanmak için sıkıştırılır; telefon galerisindeki ve internetteki görüntülerin hepsi sıkıştırılmış biçimdedir. Bugün çoğunlukla JPEG ve JPEG 2000 kullanılır. Yazarlara göre bunlar her görüntü türü için en uygun değildir ve kayıplı sıkıştırmanın ne kadar ileri gidebileceği kuramsal olarak belli değildir. Derin öğrenme birçok görüntü işleme görevinde başarılı olduğu için görüntü sıkıştırmada da başarılı olması beklenir; bu alanda genelde otokodlayıcı kullanılır. PCA, niceleme, entropi kodlaması ve LSTM gibi yöntemler kendi başına sıkıştırma yöntemi olmak zorunda değildir; sıkıştırmadan önce ya da sonra kullanılıp verimi artırır. Çalışma bunlardan LSTM dışındakileri kullanır.

5.2 Model

Çalışmanın katkısı, görüntüyü daha düşük boyutta temsil etmek için simetrik bir evrişimli otokodlayıcı (CAE) önermesi ve otokodlayıcının çıktısını daha da sıkıştırmak için ardından sırasıyla PCA döndürmesi, niceleme ve entropi kodlayıcı kullanmasıdır.

Akış şöyledir: görüntü gelir, pikselleri 0-1 aralığına normalize edilir, CAE'nin kodlayıcısından geçer. Kodlayıcıda her seferinde düzgün (uniform) gürültü eklenir. Örneğin 100 boyutlu bir girdi 10 boyutlu bir çıktıya iner; bu zaten bir sıkıştırmadır. Ardından PCA döndürmesi, niceleme ve entropi kodlaması uygulanır.

Soru: Pikseller 0-1 arasına getiriliyor dendi, ama renkli bir görüntüde (RGB) her piksel 0-255 arasında değer alır. Bu nasıl yapılıyor? **Cevap:** Bütün değerler 255'e bölünür; kod çözücüde 255 ile çarpılarak geri elde edilir. Bunun nedeni derin öğrenme modellerinin 0-1 aralığındaki değerlerle daha iyi çalışmasıdır.

5.3 Otokodlayıcının çalışması

Sunan öğrenci otokodlayıcıyı sınıflandırma yapan bir ağla karşılaştırarak anlattı. Sınıflandırmada girdi görüntüdür, çıktı bir etikettir (örneğin köpek için 0, kedi için 1); ağ, 100 piksellik girdiye 0 ya da 1 demeyi öğrenir ve hata yaptıkça ağırlıklar güncellenir. Otokodlayıcıda ise girdi ve çıktı aynıdır: ağ, görüntünün kendisini yeniden üretmeyi öğrenir.

Bunun anlamlı olmasının nedeni, katmanların ortada daralmasıdır: 100 sayılık girdi önce 80'e, sonra 50'ye iner; ardından simetrik bir yapıyla tekrar 100'e çıkar. Ağın 50 sayıdan 100 sayıyı yeniden üretebilmesi gerekir; demek ki bu 50 sayı görüntüyü temsil etmeye yeter. Ortadaki boyutu tasarımcı belirler. Eğitim tamamlandıktan sonra ağ "makasla keser gibi" ortadan ikiye ayrılır: ilk yarı kodlayıcıdır, ortadaki çıktı sıkıştırılmış veridir; ikinci yarı kod çözücüdür ve sıkıştırılmış veriden görüntüyü üretir.

5.4 PCA döndürmesi

PCA aslında bir eksen döndürme işlemidir; hoca bunu "uzayı değiştirme" diye adlandırmanın daha anlamlı olacağını söyledi. Veri, varyansın en yüksek olduğu yönlere göre yeni eksenlere taşınır. Makine öğrenmesinde PCA genelde boyut düşürmek için kullanılır, ama burada bütün bileşenler korunur; amaç daha fazla sıfır elde etmektir. Bu işlem kayıpsız olarak geri alınabilir.

5.5 Niceleme

Slayttaki formül (ekrandan, 01:00:00):

$$y' = \left[2^{B-1} \cdot y_{rot}\right]$$

Burada B istenen hassasiyet için bit sayısıdır ve modelde 12'dir; köşeli parantez yuvarlamayı gösterir. Kod çözücüde ters niceleme yapılır:

$$\tilde{y} = \frac{y'}{2^{B-1}}$$

Slayttaki örnek: $y_{rot} = 0{,}123$ olsun.

- Niceleme: $0{,}123 \cdot 2^{11} = 251{,}904$, yuvarlanınca 252.
- Ters niceleme: $252/2^{11} = 0{,}123046875$ (python ile doğrulandı).

Başlangıçtaki $0{,}123$ değeri $0{,}123046875$ olmuştur: çok yakın, ama bir kayıp vardır. Nicelemenin nedeni, buna yakın bütün ondalık değerlerin aynı tamsayıya (252) düşmesidir; entropi kodlayıcı aynı değerler çok tekrarlandığında daha verimli çalışır. Niceleme hatasını tolere edebilmesi için eğitimde her görüntüye bu hata kadar düzgün gürültü eklenir; gürültü her epokta farklı olduğu için ağ aynı görüntüyü farklı biçimlerde görür.

Son aşamada entropi kodlayıcı kullanılır. Sunan öğrenci, derste görülen Huffman ve aritmetik kodlamanın birer entropi kodlayıcı olduğunu, bu çalışmada JPEG 2000'de kullanılan entropi kodlayıcının kullanıldığını söyledi.

5.6 Sonuçlar

Görsel karşılaştırma ve süre tablosu (ekrandan, 01:01:35):

Görüntü	(a) Ham	(b) JPEG	(c) JPEG 2000	(d) CAE
Çocuk yüzü	24 bpp	0,290 bpp	0,297 bpp	0,293 bpp
Papağan	24 bpp	0,283 bpp	0,300 bpp	0,294 bpp
Çiçek	24 bpp	0,318 bpp	0,299 bpp	0,295 bpp

Kodlayıcı	Ortalama süre (s)
JPEG	0,39
JPEG2000	0,59
Ballé'nin çalışması (CPU)	7,39
Önerilen CAE (CPU)	2,29
Önerilen CAE (GPU)	0,67

Yani 24 bit/piksellik görüntüler yaklaşık 0,29 bit/piksele indirilmiştir. Yazarlar, kayıplı sonucun kendi yöntemlerinde daha iyi göründüğünü söyler. Karmaşıklık açısından çok benzer bir çalışma olan Ballé'nin çalışmasıyla karşılaştırılmıştır. JPEG'in daha hızlı olmasının nedeni olarak JPEG'in C++ ile, önerilen yöntemin Python ile yazılmış olması gösterilmiştir. Kodak veri setinde JPEG 2000'e göre %13,7 BD-rate azalması sağlanmıştır. Gelecek çalışma olarak otokodlayıcının yanında üretken çekişmeli ağların (GAN) kullanılması düşünülmüştür.

5.7 Hocanın yorumu

Hoca, önerilen yöntemin JPEG 2000'in ilerisine geçtiğini anladığını söyledi ve örnek görüntüleri sütun sütun inceledi. Çiçekli görüntüde pencerenin kenarındaki tuğlalarda C sütunu (JPEG 2000) gözüne daha iyi göründü. Papağanın sol taraftaki beyaz bölgeleri biraz tuhaf geldi. Çocuk yüzünde orijinalle fark göremediğini, orijinalin biraz daha parlak olduğunu söyledi. Sıkıştırma oranını veren bir tablo olup olmadığını sordu; sunan öğrenci, tablo olmadığını, oranın yalnız metinde %13,7 BD-rate olarak verildiğini söyledi.

> **Ek bilgi (derste anlatılmadı):** BD-rate (Bjøntegaard delta rate), iki kodlayıcının hız-bozulma eğrileri arasındaki ortalama bit hızı farkını yüzde olarak verir. Negatif ya da "azalma" olarak verilen değer, aynı kalite için daha az bit harcandığını gösterir.

6. Sunum 5: FOTA için Program Kodu Sıkıştırma (CodeComp)

Beşinci sunumda bir öğrenci, hocanın belirttiğine göre 2010 yılına ait, FOTA için hızlı bir kod sıkıştırma yöntemi öneren çalışmayı anlattı.

6.1 FOTA

FOTA (Firmware Over The Air), cihaz yazılımının kablosuz olarak güncellenmesine izin veren teknolojidir; cep telefonlarına gelen işletim sistemi güncellemeleri buna örnektir. Uygulamalar, aygıt sürücüleri, ara yazılım ve işletim sistemi telefonun flash belleğine yerleştirilir.

Güncelleme süreci, üreticinin yazılımın yeni bir sürümünü (hata düzeltmesi, özellik ekleme ya da kaldırma) hazırlamasıyla başlar. Bir delta üreteci, iki sürümü karşılaştırır ve aralarındaki farkı içeren bir delta oluşturur. Güncelleme paketi cihaz tarafından istenebilir ya da sunucu tarafından otomatik gönderilebilir; cihazdaki güncelleme aracısı mevcut yazılımı yenisiyle değiştirir. Bu sırada cihaz kullanılamaz, bu yüzden işlemin olabildiğince hızlı olması gerekir.

Maliyeti düşürmek için cep telefonlarında NAND flash belleklerde sıkıştırılmış program kodu saklanır; kod çalıştırılmadan önce RAM'e yüklenir. Yükleme süresi, sıkıştırılmış sayfanın okunma süresi ile kodunun çözülme süresinin toplamıdır. Bu yüzden özellikle talep üzerine sayfa yükleme için çok hızlı kod çözme gerekir.

Makale, program kodu için hızlı, sözlük tabanlı bir sıkıştırma algoritması önerir. Sunan öğrenciye göre kod çözme zlib'den 4 kat, kodlama ise daha önce bu alanda kullanılan BPE'den 6 kat daha hızlıdır. Ayrıca deltadaki kopya bilgileri kullanılarak, delta boyutu artırılmadan güncelleme sırasındaki sıkıştırma süresi azaltılır.

6.2 Önerilen yöntem: ana hat

Slayta göre (ekrandan, 01:10:00) önerilen yöntem, sözlük tabanlı yöntemlerin Lempel-Ziv (LZ) sınıfının bir varyasyonudur ve makalede CodeComp olarak adlandırılır. Geleneksel LZ ailesinde birim sözcük bir byte'tır ve kodlama ile kod çözme birçok bit uzunluğunda işlem içerir. CodeComp, daha yüksek oran ve daha hızlı kod çözme için çok daha az bit işlemi gerektiren bir kodlama şeması kullanır.

Temel fikir şudur: bir talimat w, referans alınan talimat r'den farklı olduğunda w yerine $\mathrm{diff}(w, r) = w \oplus r$ farkı kodlanır. Hızlı kod çözmek için fark, tek tip uzunlukta bir kodla yazılır. En sık görülen farklar büyük olasılıkla eşit dağılacağından tek tip uzunlukta kodlama sıkıştırma oranını önemli ölçüde düşürmez.

6.3 Biçimler ve kod çözme

Hızlı kod çözme için değişken uzunlukta kodlar ve çalışma uzunluğu kodlaması (RLE) kullanılmaz. Sunan öğrenci, bütün işaretler eşit olduğunda sabit sayıda işaretin tek bir işarete sıkıştırılmasını derste görülen RLE mantığına benzetti.

Kodlama bir makro blok düzeninde yapılır. Her makro bloğun referans bloğa olan bir uzaklığı (dist) vardır. Bu uzaklık bir önceki uzaklıkla aynıysa dist işareti 1 yapılır; değilse dist işareti 0 yapılır ve uzaklık dist tablosunda saklanır. Her talimat referans talimatla karşılaştırılır:

- Eşitse kopya işareti 1 yapılır.
- Eşit değilse ve fark DiffTable'da varsa istisna işareti 0 yapılır ve tablonun indeksi saklanır.
- Eşit değilse ve fark tabloda yoksa istisna işareti 1 yapılır ve değerin kendisi saklanır.

DiffTable en sık 256 farktan oluşur, bu yüzden indeksi 1 byte'tır.

Kodlama örneği (ekrandan, 01:13:10):

```
Referans sırası : 00 0F 01 02 | 21 5F | 11 F1 F2 0A 00 00 FF FE 00 01 FF F0
Kodlama sırası  : 00 0E F0 02 | 11 F1 F2 0A 00 00 FF FE 00 01 FF F0

Dist           : d1 d2
Dist işareti   : 0, 1, 0, 1, 1, 1, 1, 1
Kopya işareti  : 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1
İstisna işareti: 0, 1
İndeks         : p, öyle ki DiffTable[p] = 0F (+) 0E
İstisna değeri : F0
```

Adım adım:

- 1. byte: 00 = 00, kopya işareti 1.
- 2. byte: 0E ile 0F farklıdır; kopya işareti 0. Fark $0F \oplus 0E = 01$ DiffTable'da bulunduğu için istisna işareti 0 olur ve indeks p saklanır.
- 3. byte: F0 ile 01 farklıdır; kopya işareti 0. Fark tabloda olmadığı için istisna işareti 1 olur ve değer F0 olduğu gibi saklanır.
- 4. byte ve sonrası: kodlama sırasındaki 11 F1 F2 … bloğu referans sırasında aynen vardır (21 5F atlanarak); bu byte'ların hepsinde kopya işareti 1'dir.

Sağdaki tabloya göre (Tablo 1) CodeComp'un kod çözmesi ARM9 mimarisinde, 200 MHz ve 600 MHz saat hızlarında zlib'den yaklaşık 4 kat hızlıdır.

6.4 Kodlama şemaları

Slayttaki örnek (ekrandan, 01:15:00), "geriye doğru kodlama" içindir:

Adres	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30
Yönerge	0F	FF	FA	F3	BF	BD	FA	E2	0F	FF	0F	FF	FA	E3	BF	BD

Adres 20-30'daki 0F FF FA E3 BF BD kodlanacak olsun. İki olası yol vardır:

- **Kodlama şeması 1:** (20-22)'deki 0F FF, (16-18)'deki 0F FF'yi; (24-26)'daki FA E3, (12-14)'teki FA E2'yi; (28-30)'daki BF BD, (8-10)'daki BF BD'yi referans alır.
- **Kodlama şeması 2:** (20-30)'daki 0F FF FA E3 BF BD bütün olarak (0-10)'daki 0F FF FA F3 BF BD'yi referans alır.

İki şemada da yalnız E3 farklı bir değerle karşılaştırılır: şema 1'de E2 ile, şema 2'de F3 ile. Bu fark üzerinden istisna ve kopya işaretleriyle kodlama yapılır.

6.5 Deltadaki kopya bilgilerinin kullanılması

Delta, yeni sürümdeki bir alt dizinin eski sürümdeki bir alt dizinin kopyası olup olmadığını belirler. Kopya bilgisi varsa bu bilgi doğrudan alınır; alt dizi yeniden kodlanıp delta büyütülmez. Bunun için üç koşul kontrol edilir: yeni sürümdeki her talimat için deltadaki kopya bilgisinden eski sürümdeki karşılık gelen konumun bulunması, sıkıştırılmış eski sürümdeki uzaklık bilgisinden referans talimatlarının bulunması ve kopya bilgisinden yeni sürümdeki karşılık gelen konumun bulunması. Bu koşullar sağlanmadığında yeniden sıkıştırma yapılır. Tablo 2'ye göre yeniden sıkıştırma oranı ortalama %7'nin altındadır.

6.6 Deneysel sonuçlar

Yöntem, 600 MHz'de çalışan ARM9 mimarili bir cep telefonunda uygulanmıştır. FOTA güncellemesi altı aşamadan oluşur:

- 1. Eski sıkıştırılmış sürümün okunması
- 2. Eski sürümün kodunun çözülmesi
- 3. Deltanın uygulanması
- 4. Yeni sürümün kodlanması
- 5. Yeni sıkıştırılmış sürümün yazılması
- 6. Doğrulama

2. ve 4. aşamalar dışındakilerin süresi sisteme ve FOTA çözümüne bağlı olduğu için yalnız kod çözme ve kodlama performansı analiz edilmiştir. Delta boyutu büyük olduğunda kodlama süresi güncelleme süresini önemli ölçüde etkiler.

Sunan öğrenciye göre BPE ile CodeComp arasında kod çözme hızında yaklaşık 3 kat fark vardır; BPE daha yeni bir mimaride (ARM11) denenmesine rağmen CodeComp eski ARM9 mimarisinde ondan hızlıdır.

Kodlama performansı ve delta boyutu (ekrandan, 01:20:00):

Yöntem	Normal kodlama hızı	Delta ile kodlama hızı
BPE	60,70 KB/s	364,30 KB/s
Code Comp	379,22 KB/s	654,02 KB/s

Yöntem	Orijinal dosya	İlk delta	Artan delta	Toplam delta
BPE	62,141	740	1400	2140
Code Comp	72,744	838	0	838

Deltasız kodlamada CodeComp BPE'den $379{,}22/60{,}70 \approx 6{,}2$ kat, deltayla $654{,}02/364{,}30 \approx 1{,}8$ kat hızlıdır. BPE tabanlı yöntem ilk deltayı (740) büyütüp toplam 2140'a çıkarırken CodeComp deltayı artırmaz (python ile doğrulandı).

> **Ders sonrası not:** Slaytta BPE'nin delta boyutunu "%200" artırdığı yazıyor; tablodaki değerlerle artış $1400/740 \approx \%189$'dur, yani "%200" büyük olasılıkla yuvarlanmış bir değerdir. Ayrıca sunan öğrenci sonuç kısmında BPE'yi "byte-shiftli programlama" diye açıkladı; slayttaki kısaltma BPE, sunumun başında da söylendiği gibi Byte Pair Encoding'dir.

6.7 Sonuç

Makale, sıkıştırılmış FOTA için CodeComp adlı verimli bir kod sıkıştırma yöntemi önerir. Bit işlemlerini en aza indiren yeni biçimler tanıtılmıştır. Yöntem zlib'den 4 kat hızlı çalışır ve sıkıştırma oranı zlib'den yaklaşık %2 daha iyidir. Kod çözme, kodlama ve sıkıştırma performansının hepsi BPE tabanlı FOTA yönteminden üstündür. Deltadaki kopya bilgilerinin kullanılmasıyla kodlama süresi delta boyutu artırılmadan azaltılabilir. Yazarlara göre bu yaklaşım LZ sıkıştırma algoritmalarının çoğuna uygulanabilir.

> **Ek bilgi (derste anlatılmadı):** Byte Pair Encoding, metindeki en sık geçen bitişik byte çiftini kullanılmayan bir byte ile değiştirip bunu tekrarlayan basit bir sözlük yöntemidir. Kod çözmesi çok hızlıdır, ama her adımda en sık çifti aramak gerektiği için kodlaması yavaştır; bu, tablodaki düşük normal kodlama hızını açıklar.

7. Sunum 6: Küçük Mobil Cihazlarda Shannon-Fano ve Huffman

Altıncı sunumda bir öğrenci, hocanın belirttiğine göre 2017 yılına ait, Shannon-Fano ve Huffman algoritmalarının küçük mobil cihazlardaki metin dosyası sıkıştırma performansını inceleyen çalışmayı anlattı.

7.1 Motivasyon ve akıllı kartlar

Veri boyutları büyüdükçe depolama alanı yetersiz kalır. Bilgisayarlarda yeni disk eklemek bir çözümdür, ama akıllı telefonlar, mobil cihazlar ve akıllı kartlar gibi cihazların kapasitesi artırılamayabilir. Bu yüzden sıkıştırma yöntemleri incelenir. Bilinen yöntemlerin hepsi küçük cihazlarda kullanılamaz, çünkü bu cihazların kaynakları ve işlem gücü yetersizdir. Makalenin amacı, küçük mobil cihazlarda depolama alanını etkili kullanmak için hafif algoritmaları incelemek, Huffman ve Shannon-Fano'yu karşılaştırıp bir öneride bulunmaktır.

Makalede "küçük mobil cihaz" ile kastedilen genelde akıllı kartlardır. Sunan öğrenci, makalede ayrıntı verilmediği için akıllı kartlar hakkında kendi araştırmasını da aktardı. Akıllı kart (çip kart, entegre devre kart), üzerinde manyetik şerit, çip, barkod ya da temassız radyo frekansı vericisi gibi teknolojiler taşıyan ve bir okuyucuyla eşleştiğinde bilgi alışverişi yapabilen plastik karttır; kredi kartı, SIM kart ve kimlik doğrulama kartları örnektir. Sunan öğrenciye göre ilk örnekleri 1960'ların sonuna dayanır, 2020'de dünyada yaklaşık 10 milyar akıllı kart vardır ve bu kartlar için beş ISO standardı oluşmuştur. Kart sayısı çok büyük olduğu için standartların dışına çıkılmaz.

Akıllı kartlarda veri iki yöntemle saklanabilir:

- **XML:** Metinsel bilgiyi belli bir hiyerarşide tutan, kendini tanımlayan bir dildir. Veri gösterimi, depolama ve özellikle veri değişiminde yaygındır; sunan öğrenciye göre akıllı kartlarda XML'in tercih edilmesinin temel nedeni veri değişiminin hızlı yapılabilmesidir. Efficient XML gibi türevleri de vardır.
- **PicoDBMS:** Akıllı kartlar için tasarlanmış özel bir veri tabanıdır. Amacı veri yapılarının boyutunu küçültmek, yazma işlemlerini azaltıp okuma hızını artırmak ve belleğe doğrudan erişim sağlamaktır. Üç depolama yöntemi kullanır: Flat Storage, Domain Storage ve Ring Storage.

Sunan öğrenciye göre akıllı kartların kaynakları çok kısıtlıdır (yaklaşık 1 MB RAM ve 256 KB depolama düzeyinde).

7.2 Benzer çalışmalar: Burrows-Wheeler Transform

Slayttaki bilgiler (ekrandan, 01:30:00):

- BWT kayıpsız bir algoritmadır ve metin sıkıştırmada hız/performans açısından en iyi algoritmalardan biridir.
- En büyük dezavantajı her veride sıkıştırma yapamamasıdır, çünkü performansı giriş verisinin karakteristiğine bağlıdır. Sunan öğrenciye göre girdide benzer örüntüler ne kadar çoksa performans o kadar iyidir.
- BW algoritmasının üç temel iterasyonu vardır: BWT, Move-to-Front (MTF) ve entropi kodlayıcı (Huffman, aritmetik kodlama, Golomb, Fibonacci kodlama vb.).
- BWT'yi MTF ile birleştirmek performansı önemli ölçüde iyileştirir; Huffman ve aritmetik kodlamayla birleştirmek de iyi sonuç verir.
- BWT'nin birçok varyasyonu vardır; farklar genelde 2. ve 3. aşamadaki algoritmaların değiştirilmesinden gelir. Bu dönüşümlerden sonra RLE kullanılarak daha iyi sonuç alınır.
- Metin bir ön işlem adımından da geçirilebilir; ön işlemde genelde sözlük tabanlı yöntemler kullanılır.

Sunan öğrenciye göre BWT'nin ele alınmasının nedeni hafif bir algoritma olması ve küçük cihazların kaynak gereksinimleriyle örtüşmesidir. Literatürde BWT ile aritmetik kodlamanın birleştirildiği çalışmalar ve sinir ağı tabanlı çalışmalar da vardır. Sinir ağları sıkıştırma oranı açısından çok iyidir, ama sıkıştırma süresi ve kaynak kullanımı akıllı kartlar için uygun değildir.

7.3 Deney düzeni

Ölçütler sıkıştırma süresi ve sıkıştırma oranıdır. Test verisi, ASCII metinlerden oluşan Calgary Corpus'tur. Corpus'tan boyutları 36 KB ile 1 MB arasında değişen 5 metin alınmış, bunlar farklı biçimlerde birleştirilerek 5 test dosyası daha üretilmiştir. Sunan öğrenci, 1,1 MB ile 3,8 MB arasındaki bu dosyaların bilgisayar için çok küçük, ama 256 KB ya da 512 KB kapasiteli akıllı kartlar için çok büyük olduğuna dikkat çekti. Shannon-Fano ve Huffman herhangi bir dönüşüm uygulanmadan, saf halleriyle kullanılmıştır.

7.4 Sonuçlar

Dosya boyutu (ekrandan, 01:40:00):

No	Dosya	Shannon-Fano (byte)	Huffman (byte)
1	test1.txt	1941646	1908295
2	test2.txt	745249	724476
3	test3.txt	971274	960170
4	test4.txt	1165046	1145025
5	test5.txt	2568026	2548143

Sıkıştırma oranı ve kodlama süresi (ekrandan, 01:35:45):

No	Dosya	Oran SF (%)	Oran H (%)	Süre SF (ms)	Süre H (ms)
1	test1.txt	38,02	39,08	101,09	103,45
2	test2.txt	34,12	35,96	54,86	47,81
3	test3.txt	34,16	34,91	52,52	63,20
4	test4.txt	38,01	39,08	62,00	71,43
5	test5.txt	33,67	34,19	127,38	120,02

Slayttaki özet: Shannon-Fano daha hızlıdır ama sıkıştırma oranı daha düşüktür; Huffman daha yavaştır ama sıkıştırma oranı daha yüksektir. Her ikisi de orijinal veriye göre yaklaşık %35-40 sıkıştırma yapar. Bazı dosyalarda (test2, test5) Huffman daha hızlıdır. Makale her iki yöntemin de kullanılabileceğini söyler, ama Shannon-Fano'yu önerir. Gelecek çalışma olarak BWT varyasyonlarının LZ77 ile karşılaştırılması ve algoritmaların görüntü ve müzik gibi verilerde de denenmesi önerilmiştir.

7.5 Makaledeki tutarsızlık

Sunan öğrenci, Huffman'ın önerilmesini beklediğini, makaleyi ayrıntılı incelediğinde bulgular ve sonuç bölümleri arasında çelişki bulduğunu anlattı (ekrandan, 01:40:00 ve 01:41:20):

- Bulgular bölümünde: Huffman, boyut açısından Shannon-Fano'dan yaklaşık %1,66 daha iyidir; Shannon-Fano'nun sıkıştırma süresi ortalamada yaklaşık %2,3 daha iyidir.
- Sonuç bölümünde: Shannon-Fano'nun ortalama dosya boyutu Huffman'dan %63,32 daha yüksektir (fark yaklaşık %1,22); Shannon-Fano'nun oranı Huffman'dan %36,27 daha düşüktür (fark %1,12); Shannon-Fano'nun ortalama süresi Huffman'dan yüksektir: 65,11 ms ve 64,82 ms.

Tablodaki süreler 5'e bölünerek ortalama alındığında ise sonuç farklıdır:

$$\bar{t}_{SF} = \frac{101{,}09 + 54{,}86 + 52{,}52 + 62{,}00 + 127{,}38}{5} = \frac{397{,}85}{5} = 79{,}57 \text{ ms}$$

$$\bar{t}_{H} = \frac{103{,}45 + 47{,}81 + 63{,}20 + 71{,}43 + 120{,}02}{5} = \frac{405{,}91}{5} \approx 81{,}18 \text{ ms}$$

Yani tabloya göre Shannon-Fano yaklaşık %2 daha hızlıdır ve sonuç bölümündeki 65,11 / 64,82 ms rakamları tabloyla uyuşmaz (python ile doğrulandı). Sunan öğrenci bu konuda makalenin yazarına e-posta gönderdiğini söyledi.

> **Ders sonrası not:** Tablodaki oranların ortalaması Shannon-Fano için %35,60, Huffman için %36,64'tür; sonuç bölümündeki %36,27 değeri de bu ortalamalarla doğrudan eşleşmez. Dosya başına boyut farklarının ortalaması %1,63'tür; bulgular bölümündeki %1,66 değerine yakındır (python ile hesaplandı).

7.6 Hocanın soruları ve yorumu

Hoca önce hangi bölümde hangi yöntemin daha iyi denildiğini sordu; sunan öğrenci çelişkiyi makaleden göstererek açıkladı.

Hoca, Shannon-Fano'nun Huffman'dan çok kötü olmadığını, derste söylendiği gibi Shannon-Fano'nun daha yüksek başarı göstermesi için olasılıkların 2'nin kuvvetleriyle ilgili bir koşulu sağlaması gerektiğini hatırlattı. Ardından Shannon-Fano'dan önce BWT uygulamanın yardımcı olup olmayacağını düşündü, ama BWT bir dönüşüm yapsa da karakterlerin frekanslarını değiştirmediğini belirtti.

> **Ek bilgi (derste anlatılmadı):** Sembol olasılıkları $1/2, 1/4, 1/8, \ldots$ gibi 2'nin negatif kuvvetleri olduğunda hem Shannon-Fano hem Huffman ortalama kod uzunluğunu büyük olasılıkla entropiye eşitler. BWT yalnız karakterlerin sırasını değiştirdiği için sıfırıncı dereceden frekansları değiştirmez; ancak ardından gelen MTF aşaması küçük sayıların sıklığını artırarak entropi kodlayıcıya yardım eder.

Hoca, yayımlanmış makalelerde de hata olabileceğini söyledi. Büyük araştırma gruplarında herkesin projenin bir parçasını yapıp herkesin yazar olarak yer alabildiğini, bu yüzden bazı yazarların çok sayıda yayınının olabildiğini, hakemlerin de bazen yalnız yazar adına bakarak çalışmayı kabul edebildiğini belirtti. Beş sayfalık kısa bir makalede bu tür hataların yapılmaması gerektiğini ekledi. Sunan öğrencinin tabloyu hesaplayarak bu çelişkiyi yakalamasını olumlu karşıladı.

8. Sunum 7: Sualtı Nesnelerin İnterneti için DWT-CNN ile Görüntü Sıkıştırma

Son sunumda bir öğrenci, sualtı nesnelerin interneti (Internet of Underwater Things, IoUT) için derin öğrenme tabanlı bir görüntü sıkıştırma modelini İngilizce anlattı.

8.1 Problem

Nesnelerin interneti (IoT) son yıllarda uygulama alanlarını genişletmiştir. Akıllı sualtı nesnelerinin çektiği görüntülerin iletimi, sualtı ortamının doğası nedeniyle zordur; IoUT için verimli bir görüntü iletim stratejisi gerekir. Çalışma, DWT (ayrık dalgacık dönüşümü) tabanlı bir derin öğrenme modeli önerir.

8.2 DWT-CNN mimarisi

Slayttaki açıklama ve şekil (ekrandan, 01:50:00): DWT-CNN modeli iki CNN içerir; biri sualtı tarafında, diğeri yüzey istasyonundadır. Arada DWT tabanlı bir görüntü kodlayıcı kullanılır. Vericide, yani sualtındaki algılayıcı düğümlerde ya da IoUT cihazlarında, görüntü yapısal bilgiyi koruyan bir CNN ile sıkışık bir temsile dönüştürülür. İki CNN, görüntüler sıkıştırılırken daha iyi kalite elde etmek için birlikte kullanılır.

```
IoUT nesneleri -> Orijinal görüntü -> C-CNN -> Sıkışık temsil -> DWT -> Kanal
Kanal -> IDWT -> Çözülmüş görüntü -> R-CNN -> Yeniden kurulmuş görüntü
```

C-CNN (sıkıştırma CNN'i): Sunan öğrenciye göre üç katmanlıdır. Görüntüden uzamsal özellikleri çıkarır ve kodlanan görüntünün doğru geri kurulmasına yardım eder. İlk katman yama (patch) çıkarma ve temsil işini yapar.

R-CNN (yeniden kurma CNN'i) (ekrandan, 01:51:50): 20 ağırlık katmanı vardır ve üç tür katman kullanılır: Conv + ReLU, Conv + Batch Normalization + ReLU ve Conv.

- İlk katmanda $3 \times 3 \times c$ boyutunda 64 filtre ile 64 özellik haritası üretilir, ardından ReLU gelir (c görüntünün kanal sayısıdır).
- 2.-19. katmanlarda $3 \times 3 \times 64$ boyutunda 64 filtre kullanılır; Conv ile ReLU arasına toplu normalizasyon eklenir.
- Son katmanda $3 \times 3 \times 64$ boyutunda c filtre ile çıktı yeniden kurulur.
- Artık öğrenme (residual learning) ve toplu normalizasyon, eğitimi hızlandırmak ve sonucu iyileştirmek için kullanılır.
- Çözülen görüntü, bikübik ara değerlemeyle girdi boyutuna büyütülür.

8.3 Performans değerlendirmesi

Yeniden kurulan görüntünün kalitesi, bir veri setindeki görüntüler üzerinde PSNR (tepe sinyal-gürültü oranı) ile ölçülmüştür. Slayttaki grafik (ekrandan, 01:54:30), on kare (Frame_002, Frame_014, Frame_103, Frame_125, Frame_215, Frame_144, Frame_162, Frame_193, Frame_236, Frame_566) için $S = 100, 80, 60, 40, 20$ değerlerinde PSNR'ı gösterir. Görüntü kalitesi S arttıkça iyileşir. Örneğin Frame_002 için PSNR $S = 20$'de 32,29 dB iken $S = 100$'de 51,55 dB'ye çıkar. Grafikte ilk beş karede $S = 100$ için PSNR yaklaşık 51 dB, son beş karede yaklaşık 56-57 dB'dir.

8.4 Sonuç

Makale, görüntüyü sıkışık biçimde temsil etmek için DWT ile iki CNN'i kullanan verimli bir öğrenme yaklaşımı sunar. Gelecek çalışma olarak yöntemin gürültülü ortamlarda denenmesi

ve DWT-CNN sonucunu iyileştirmek için gürültü giderme (denoising) adımı eklenmesi önerilmiştir.

Hoca, makaleyi biraz karmaşık bulduğunu söyledi.

Örnek Sorular

Soru 1 (derste sözlü, 00:14:17). Bayesçi sıkıştırma çalışmasında seyreltme sırasında bazı kelimeler atılıyor. Bu çalışma bir sıkıştırma çalışması mıdır, sınıflandırma çalışması mıdır? Atılan kelimelere geri dönülebilir mi?

Çözüm (derste verildi): Hocaya göre çalışmanın amacı sınıflandırmadır. Atılan kelimelere geri dönülemez; sıkıştırma metni değil sınıflandırma modelini küçültür. Sunan öğrenciye göre atılan kelimeler sınıflandırmaya katkı vermeyen kelimelerdir ve doğruluk büyük ölçüde korunur (IMDb'de %84,1'den %83,6'ya, 12985 kat sıkıştırmayla).

Soru 2 (derste sözlü, 00:43:05). FastText tabanlı yöntemde "car" kelimesi 3 karakterdir, ama vektörü 100 boyutludur. Bu yöntem veriyi nasıl küçültür?

Çözüm (derste verildi): Vektör gönderilmez; yalnız benzerlik bulmak için kullanılır. Kelimenin kosinüs benzerliğine göre en yakın K komşusundan en az karakterli olanı ve onun sıra numarası gönderilir. Hocanın hesabıyla "automobile" 10 karakterdir, onun yerine "car" ve konum bilgisiyle yaklaşık 4 byte gider.

Soru 3 (ekrandan, 00:50:00). $K = 30$ ile "what is your phone number" cümlesi "it40is1my2fax12" olarak sıkıştırılıp "what is your phone" olarak açılmıştır. Bu sıkıştırma kayıplı mıdır, kayıp nereden gelir?

Çözüm (derste verildi): Kayıplıdır. K büyüdükçe daha uzak ve daha kısa komşular seçilebilir; açılışta orijinal kelime her zaman bulunamaz. Bu örnekte "number" kaybolmuştur. $K = 5$ ile aynı cümle kayıpsız açılır. Sunan öğrenciye göre K, kayıp oranını ayarlamayı sağlar.

Soru 4 (derste sözlü, 00:55:59). Otokodlayıcıda pikseller 0-1 aralığına normalize ediliyor. RGB görüntüde 0-255 arasındaki değerler nasıl bu aralığa getirilir ve geri alınır?

Çözüm (derste verildi): Her değer 255'e bölünür, kod çözücüde 255 ile çarpılarak geri alınır. Derin öğrenme modelleri 0-1 aralığında daha iyi çalıştığı için bu yapılır.

Soru 5 (ekrandan, 01:00:00). $B = 12$ için PCA sonrasında $y_{rot} = 0{,}123$ değerini nicemleyin ve ters nicemleyin. Kayıp ne kadardır?

Çözüm (derste verildi): $y' = [2^{11} \cdot 0{,}123] = [251{,}904] = 252$; $\tilde{y} = 252/2^{11} = 0{,}123046875$. Kayıp $|0{,}123046875 - 0{,}123| = 0{,}000046875$'tir (ara adım eklendi; python ile doğrulandı).

Soru 6 (ekrandan, 01:13:10). CodeComp örneğinde referans sırası 00 0F 01 02, kodlama sırası 00 0E F0 02 ile başlıyor. İlk dört byte için kopya ve istisna işaretlerini yazın.

Çözüm (derste verildi): 1. byte eşit, kopya işareti 1. 2. byte farklı (0E ve 0F), kopya 0; fark $0F \oplus 0E$ DiffTable'da olduğu için istisna işareti 0 ve indeks p saklanır. 3. byte farklı (F0 ve 01), kopya 0; fark tabloda olmadığı için istisna işareti 1 ve değer F0 saklanır. 4. byte eşit, kopya 1. Kopya işaretleri 1, 0, 0, 1; istisna işaretleri 0, 1.

> **Çözüm (derste verilmedi):** $0F \oplus 0E = 01$ (onaltılı), yani DiffTable'da aranan fark değeri 01'dir (python ile doğrulandı).

Soru 7 (ekrandan, 01:20:00). Tablo IV ve V'e göre CodeComp, BPE'ye göre normal kodlamada ve deltayla kodlamada kaç kat hızlıdır? BPE deltayı yüzde kaç büyütür?

> **Çözüm (derste verilmedi):** Normal kodlama: $379{,}22/60{,}70 \approx 6{,}25$ kat. Deltayla: $654{,}02/364{,}30 \approx 1{,}80$ kat. BPE'de delta 740'tan 2140'a çıkar; artış $1400/740 \approx \%189$'dur (slaytta yaklaşık %200 olarak verilmiş). CodeComp'ta artan delta 0'dır (python ile doğrulandı).

Soru 8 (ekrandan, 01:35:45). Tablodaki kodlama sürelerine göre Shannon-Fano ve Huffman'ın ortalama kodlama süresi nedir? Makalenin sonuç bölümündeki 65,11 ms ve 64,82 ms değerleri tabloyla tutarlı mıdır?

Çözüm (derste verildi): Shannon-Fano ortalaması $397{,}85/5 = 79{,}57$ ms, Huffman ortalaması $405{,}91/5 \approx 81{,}18$ ms'dir. Tabloya göre Shannon-Fano daha hızlıdır; sonuç bölümündeki değerler tabloyla tutarlı değildir (python ile doğrulandı).

Soru 9 (derste sözlü, 01:38:35). Shannon-Fano ne zaman Huffman kadar iyi sonuç verir? Shannon-Fano'dan önce BWT uygulamak sonucu iyileştirir mi?

Çözüm (derste verildi): Hoca, derste anlatıldığı gibi Shannon-Fano'nun olasılıklar 2'nin kuvvetleriyle ilgili bir koşulu sağladığında daha başarılı olduğunu hatırlattı. BWT bir dönüşüm yapar, ama karakterlerin frekanslarını değiştirmez; bu yüzden tek başına Shannon-Fano'ya yardım etmesi beklenmez.

> **Çözüm (derste verilmedi):** Olasılıklar 2^{-k} biçimindeyse iki yöntem de entropiye eşit ortalama kod uzunluğu verir.

Dersin Toparlaması

Bu derste yedi seminer sunumuyla veri sıkıştırmanın farklı uygulama alanları görüldü. İlk iki sunum sinir ağları ile sıkıştırmanın iki ayrı anlamını gösterdi: Bayesçi seyreltmede sıkıştırılan şey modelin kendisidir ve hoca bunun metin sıkıştırmadan farklı olduğunu vurguladı; Transformer çalışmasında ise sinir ağı, derste görülen aritmetik kodlamanın olasılık tahmincisi olarak kayıpsız sıkıştırma yapar. FastText ile metin kodlama, kelimeyi daha kısa bir anlamsal komşuyla değiştiren kayıplı bir aşamayı derste işlenen kanonik Huffman koduyla birleştirdi; otokodlayıcı çalışması ise derin ağ, PCA, niceleme ve entropi kodlamasını bir görüntü sıkıştırma hattında bir araya getirdi.

Son üç sunum klasik yöntemlere daha yakındır. CodeComp, LZ ailesinin farka dayalı bir varyasyonudur ve hız için değişken uzunluklu kodlardan bilerek vazgeçer. Shannon-Fano ve Huffman karşılaştırması, derste görülen iki entropi kodlayıcının pratikte çok yakın sonuç verdiğini ve yayımlanmış bir makalede bile sonuçların tablolarla karşılaştırılarak okunması gerektiğini gösterdi. DWT-CNN çalışması ise bir önceki derste anlatılan dalgacık dönüşümünü CNN tabanlı sıkıştırma ve yeniden kurma ile birleştirdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.